

XRPL Validator — Daily Engineering Report • 2026-08-12

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- **11 commits** (xrpl-validator 11)

Engineering changes

- **feat(xrpl-node): DX_VALCHECK compares EXACTLY, at STAmount precision** (adca53a, xrpl-validator)
The target is byte parity: a validator serialises the stored decimal into the SLE and hashes it, so one digit apart is a different state hash and a diverged node however small the number. The old check could not certify that — it parsed both sides into f64 and compared against a 1e-12 tolerance.
- **fix(xrpl-ledger): Number division DISCARDS its remainder at the 16-digit scale** (e7183b3, xrpl-validator)
129 -> 74 byte-parity hits, 43% of the backlog, from one line. Payment 89 -> 42, OfferCreate 29 -> 21, no key-level regressions.
- **fix(xrpl-ledger): a single-asset withdrawal pays what its adjusted tokens are worth** (bbf6ee3, xrpl-validator)
AMMWithdraw 7 -> 1 hits. #105816437 AC7713F9 and #105838164 181F8088 go fully clean; the AMM family drops 11 -> 5.
- **fix(xrpl-ledger): port rippled's Newton-iteration square root** (36a6fdb, xrpl-validator)
AMMDeposit 4 -> 0. #105779597 and #105828322 go fully clean; the AMM family drops 5 -> 1, and only a tfWithdrawAll hit remains.
- **fix(xrpl-ledger): withdraw-all pays two rounded steps, not a fused muldiv** (81180a1, xrpl-validator)
Closes the last AMM hit. #105796380 2437575D goes AMM-clean; the AMM value family is now ZERO, from 11 this morning.
- **fix(xrpl-ledger): the DIRECT walk charges the input issuer's transfer fee too** (cf36009, xrpl-validator)
#105877543 435A9AEB -> 0 hits. It is the specimen the direct walk was missing when this was attempted and reverted earlier on 2026-08-11 for want of a failing ledger.
- **fix(xrpl-ledger): multiPath prices a clamped pool fill at the OFFER's quality** (4a97124, xrpl-validator)
OfferCreate >=1000-ulp cluster 14 -> 5 hits from one rule. #105802230 EF1642A9, #105284280 BE7884AA and #105666830 A8830CA4 go fully clean; #105930662 40FB322E and #105954798 D5887FD7 each drop to one.
- **fix(xrpl-ledger): store offer residuals at STAmount precision, not exactly** (4556384, xrpl-validator)
TOffer::consume subtracts through STAmount, so a maker's remaining side comes back rounded to 16 significant digits (half-even, via Number under fixUniversalNumber). me_sub is EXACT — it keeps every digit — so whenever gives0 - give needed more than 16 digits we stored precision the ledger cannot hold and rippled does not have. Being more precise than rippled is a defect: the validator serialises the stored ...
- **fix(xrpl-ledger): charge the input transfer fee once — the walk owns it** (41ef4ac, xrpl-validator)
payment.rs and offer.rs both quoted BookStep.cpp:770 and both applied the issuer's rate, and neither

comment knew the other existed. `measure_hop` GRANTS a balance and differences it, and the walk debits the fee per fill, so the fee was already inside `consumed` before the reverse pass grossed it up again — the rate compounded.

- **fix(xrpl-ledger): price bridged fills at the filed BookDirectory rate** (`e2a9c99`, `xrpl-validator`) An offer carries two rates and they are not the same number: the rate it is FILED under (low 64 bits of `BookDirectory`, 16-digit mantissa, fixed at creation) and its current `TakerPays` / `TakerGets`, which drifts every time a residual is re-rounded. rippled prices a PARTIAL fill at the filed rate and a FULL one at the offer's own amounts. The direct walk has done this since `full_offer`; `cross_bridged` priced both ...
- **fix(xrpl-ledger): reprice a rested remainder at the ENCODED ratio** (`070ca31`, `xrpl-validator`) Two changes to `offer.rs`; only the second moves any ledger.

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **Commit-Reveal Randomness Beacon for Draws** (gaming) Add a new reserve-bearing `ltRANDOM_BEACON` object (`keylet::randomBeacon(account, seq)`) created by `RandomCommit`, which stores `sfCommitHash = sha512Half`
- **Multilateral Netting Settlement Batches** (settlement) Add a new `ttNETTING_SETTLE` transaction that carries a bounded (max 8, mirroring Batch's inner-transaction cap) `STArray` of pre-signed Payment 'obligati
- **Structured Travel-Rule Payload on Checks** (compliance) Add an optional, size-bounded `sfCompliancePayload` `SField` (VL blob, max ~512 bytes) to `CheckCreate` and `CheckCash`, carrying FATF originator/beneficiary

Appendix — commits

Time	Repo	Commit	Subject
00:00	xrpl-validator	<code>adca53a</code>	feat(xrpl-node): <code>DX_VALCHECK</code> compares EXACTLY, at <code>STAmount</code> precision
02:22	xrpl-validator	<code>e7183b3</code>	fix(xrpl-ledger): Number division DISCARDS its remainder at the 16-digit scale
09:37	xrpl-validator	<code>bbf6ee3</code>	fix(xrpl-ledger): a single-asset withdrawal pays what its adjusted tokens are worth
10:36	xrpl-validator	<code>36a6fdb</code>	fix(xrpl-ledger): port rippled's Newton-iteration square root
11:23	xrpl-validator	<code>81180a1</code>	fix(xrpl-ledger): withdraw-all pays two rounded steps, not a fused muldiv
12:10	xrpl-validator	<code>cf36009</code>	fix(xrpl-ledger): the DIRECT walk charges the input issuer's transfer fee too
14:36	xrpl-validator	<code>4a97124</code>	fix(xrpl-ledger): multiPath prices a clamped pool fill at the OFFER's quality
16:58	xrpl-validator	<code>4556384</code>	fix(xrpl-ledger): store offer residuals at <code>STAmount</code> precision, not exactly
18:38	xrpl-validator	<code>41ef4ac</code>	fix(xrpl-ledger): charge the input transfer fee once — the walk owns it
18:38	xrpl-validator	<code>e2a9c99</code>	fix(xrpl-ledger): price bridged fills at the filed BookDirectory rate
23:55	xrpl-validator	<code>070ca31</code>	fix(xrpl-ledger): reprice a rested remainder at the ENCODED ratio